Who is this course for?

Welcome to the course!

This course is for anyone who has a solid understanding of CSS and wants to take their coding to the next level. You don't need a formal degree or even any kind of background in technology to get started. All you need is the determination to learn!

By understanding CSS, I mean you should have experience using CSS to style HTML elements. Ideally, you'll have built a few web pages and be comfortable doing so. In saying that, it's not a strict prerequisite, but you'll find it much easier to get going with this knowledge.

So, if you're ready to learn all about SASS (short for Syntactically Awesome Stylesheets), you've come to the right place! We'll be going from beginner to advanced level. And by the end of the course, you'll be writing code that's faster, more efficient, and much easier to maintain.

So without further ado, let's get started!

An Introduction to CSS pre-processing

In this lesson, we'll introduce the concept of CSS pre-processing, and take a look at how it benefits us as front-end developers.

WE'LL COVER THE FOLLOWING

- Defining CSS Preprocessors
- Why use preprocessors?

Welcome to Learning SASS from scratch!

My aim in this course is to get you from beginner to expert, with a solid practical understanding to utilize in real-world projects!

Before taking our deep dive into all things SASS, I'd like to begin by providing a little context.

SASS (or Syntactically Awesome Style Sheets) is a tool known as a **CSS preprocessor**. CSS preprocessors are scripting languages that extend the default capabilities of CSS.

Other popular examples include LESS and Stylus. They are in fact all great tools to utilize when you wish to code more maintainable CSS. Especially when working with larger codebases.

Defining CSS Preprocessors

Defining CSS Preprocessors

"A CSS preprocessor is a program that lets you generate CSS from the preprocessor's own unique syntax. There are many CSS preprocessors to choose from, however most CSS preprocessors will add some features that don't exist in pure CSS, such as mixin, nesting selector, inheritance selector, and so on. These features make the CSS structure more readable and easier to maintain". — MDN

Once you're comfortable with CSS, the next natural step is to utilize a preprocessor. The biggest advantage is not having to repeat yourself.

This advantage is known as keeping your CSS **Dry**, which stands for *"Don't Repeat Yourself."*

Why use preprocessors?

The primary advantages are as follows:

- Cleaner code thanks to variables and nesting.
- Greater ease of maintenance and organisation as we can separate our files into modules.
- Ability to implement logic and calculation in our stylesheets.
- Overall improvement in workflow, which will **save hours of development time!**

Of all the preprocessors, **SASS** is by far the most popular within the developer community.

So let's get Sassy!

What is SASS?

In this introduction, we'll define SASS and its relationship to SCSS (aka Sassy CSS).

WE'LL COVER THE FOLLOWING

- What is SASS?
- SCSS or Sass?

What is SASS?

SASS is a scripting language that provides us with features and tools that regular CSS doesn't have.

Using SASS we can write more readable, maintainable and reusable code. Think of it as an extension that adds power and elegance to CSS.

It gives us various features such as:

- Variables
- Nesting
- Mixins
- Functions
- Partials & Imports
- Inheritance (Extend functionality)
- Control directives

In other words, SASS helps us organize large stylesheets in a more maintainable way.

And we'll be looking into all of these features throughout this course!

SCSS or Sass?

In SASS there are **two** different syntaxes: **SCSS** and **Sass**. However after being compiled they generate similar output.

SCSS (aka Sassy CSS) is the modern standard. It's syntax is very similar to CSS in that it uses brackets and semicolons. Even normal CSS is valid in this syntax. The file extension is `.scss`.

Sass is an older syntax that focuses on indentation to separate code blocks and newline characters to separate rules. It has the file extension `.sass`.

Throughout this course, I will use **SCSS** as it's the more natural syntax (when we use SCSS we still call it SASS!).

It's also really easy to convert regular CSS to SCSS, as you can just paste in the CSS and work from there!

Note: To see how to install and configure SASS on a local machine, check out the lesson titled "How to Install and Configure SASS Locally" in the appendix.

Next up, let's take a deep dive into the features of SASS!

Variables

In this lesson, we'll be introducing the concept of SAS S variables.

WE'LL COVER THE FOLLOWING

- Definition
- Example

SASS effectively gives us a lot of the programmatic benefits of working with code, only now with the ability to apply it to stylesheets.

Over the next few lessons we'll be diving right into the features of SASS.

First up let's introduce **variables**.

Definition

Variables are a way to store information that you want to reuse throughout your stylesheet.

They allow us to store values for colors, fonts or really any CSS value that you want to reuse!

We use the `$` symbol when we wish to make something a variable.

Example

In our SCSS, let's define a color variable:

```
$color-primary: #ffff00; // Yellow

body {
  background-color: $color-primary;
}
```

This will of course, set our `background-color` to yellow. It's that simple!

Note: You can use single line comments in Sass with `//`.

When we then run our compile*, it'll output the following CSS:

```
body {
  color: #ffff00;
}
```

This becomes extremely powerful when working on large projects!

If you wish to make a change to a colour used throughout your stylesheets, it's much simpler to alter if the color is defined in one location as a single variable.

The alternative to changing the value of one variable defined at one location is finding and replacing every reference to the value you want to change. This is a much more tedious task, especially if you want to implement a quick change to test out a different color or font.

Next up let's look at the nesting syntax!

*Later in the course we'll examine this process in detail. For now it's good to know that when we save our code into `sass/main.scss`, it'll automatically compile into the `css/style.css` file!

Nesting

In this lesson, we'll be introducing the SASS nesting syntax.

WE'LL COVER THE FOLLOWING

- Definition
- Example

When you observe the structure of an HTML file, you'll notice it has a very clear hierarchy:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
<body>

<h1>My First Heading</h1>
<p>My first paragraph.</p>

</body>
</html>
```

As you can see, HTML has a structure that makes it quite easy to read.

CSS, on the other hand, lacks this visual structure. Which is why it has a tendency to become disorganized quite quickly. Enter Sass **nesting**!

Definition

Using nesting, we can nest child selectors inside of the parent selector.

This results in much cleaner and less repetitive code.

Example

Example

Take the following HTML:

```html
<nav class="navbar">
  <ul>
    <li>Home</li>
    <li>Store</li>
    <li>Contact Us</li>
  </ul>
</nav>
```

Using regular CSS, we would write this like so:

Output

HTML

CSS (SCSS)

```scss
.navbar {
  background-color: orangered;
  padding: 1rem;
}
.navbar ul {
  list-style: none;
}
.navbar li {
  text-align: center;
  margin: 1rem;
}
```

There's a lot of repetition here. Each time we want to style a child of `navbar`, we have to repeat the class name.

With Sass, we can write much cleaner code.

Like so:

Output

HTML

CSS (SCSS)

```scss
.navbar {
  background-color: orangered;
```

```
  padding: 1rem;
  ul {
    list-style: none;

  }
  li {
    text-align: center;
    margin: 1rem;
  }
}
```

Using indentation, you can now see the `ul` and `li` selectors are neatly nested inside the `navbar` selector.

We have a much less repetitive syntax, which is far easier to read! Working with stylesheets is about to get fun!

Next up, we'll be looking at mixins.

Mixins

In this lesson, we'll be looking at the SASS mixin syntax.

WE'LL COVER THE FOLLOWING

- Definition
- Example

Definition

Another powerful feature of Sass are **mixins**. Using mixins, you can group together multiple CSS declarations for reuse throughout your project.

Example

Say we want to create a mixin to hold the vendor prefixes for a transform property.

in SASS, we'd code it like so:

```
@mixin transform {
  -webkit-transform: rotate(180deg);
  -ms-transform: rotate(180deg);
  transform: rotate(180deg);
}
```

To add the mixin into our code, we then use the `@include` directive, like so:

```
.navbar {
  background-color: orangered;
  padding: 1rem;
  ul {
    list-style: none;
  }
  li {
```

```
    text-align: center;
    margin: 1rem;
    @include transform;
  }
}
```

Note: Don't worry too much about where to put your mixin code for now. This will be explained in the next chapter. For now, just take note of the general idea!

All the code in the transform mixin will now be applied to the `li` element. You can also pass values into your mixins to make them even more flexible.

Instead of adding a specified value, add a name (using a variable, like `property`) to represent the value like so:

```
@mixin transform($property) {
  -webkit-transform: $property;
  -ms-transform: $property;
  transform: $property;
}
```

Now we can pass in whatever value we like, whenever we call the mixin:

```
@include transform (rotate(20deg));
```

In our next lesson, we'll take a look at the function syntax.

Functions

In this lesson, we'll be looking at how to write functions in SASS.

WE'LL COVER THE FOLLOWING

- Definition
- Example

Definition

Sass **functions** can receive arguments and return a single value.

They add an element of programming to writing CSS code, and we can now do math!

The standard math operators `+`, `-`, `*`, `/`, and `%` can all be utilized.

Example

The following function can accept two arguments, `$first-number` and `$second-number`. The value that is returned by the function is the sum of the two variables:

```
@function add-numbers($first-number, $second-number) {
  @return $first-number + $second-number
}
```

Say we want to replace the value of a `padding` property with the sum of two separate values.

We would call our function and pass in the arguments like so:

```
.box1 {
  padding: add-numbers(5px, 10px);
}
```

The resulting CSS output would be:

```
.box1 {
  padding: 15px;
}
```

Try it out below!

Output

HTML

CSS (SCSS)

```
@function add-numbers($first-number, $second-number) {
  @return $first-number + $second-number
}

.navbar {
  background-color: orangered;
  padding: add-numbers(5px, 100px);
  ul {
    list-style: none;
  }
  li {
    text-align: center;
    margin: 1rem;
  }
}
```

As you can see, functions help you write more readable and DRY Sass, as you can utilize reusable logic in a very efficient manner. This can make a huge difference when you start working on larger and more complex projects!

Next up, we'll be looking at Partials and Importing.

Partials and Importing

In this lesson, we'll be looking at how to modularize our files with partials & importing.

WE'LL COVER THE FOLLOWING

- Definition
- Example
- @import in CSS vs SASS

Definition

Partials in SASS help us to break up our files into smaller files without affecting performance.

The use of partials allows us to modularize our CSS, to help keep things maintainable.

We divide up our Sass into separate files representing different components. A partials' name will always start with an underscore `_`. We then import the partial using an `@import` directive.

Example

Let's say our Sass file is getting rather large. We might choose to make a partial file that contains just the code that's relevant to the header section; we'd call it `_header.scss` and move the appropriate code into that new file.

We would then import it back into `main.css`, like so:

```
// In main.scss
@import 'header';
```

Note: When you import a file, there's no need to include the `_` or `.scss` file extension.

@import in CSS vs SASS

The `@import` directive is of course, also available in CSS. However, there is an important difference. Whenever an import statement is used in CSS, an HTTP request is made to the server. This increases the amount of resource requests and negatively affects the performance of a web page. SASS does not do that. SASS takes the file that you want to import from and combines it with the file you're importing. So you can serve a single CSS file to the web browser, which does not affect the performance.

In the next lesson, we'll be looking at how to implement inheritance with `extends`.

Inheritance and Extends

In this lesson, we'll be looking at how to implement inheritance with the extends directive.

WE'LL COVER THE FOLLOWING

- Definition
- Example

Definition

Another great feature of Sass is **inheritance**. We implement this using the `@extend` directive.

Inheritance is a feature of SASS that allows multiple classes to share a common set of properties with one another.

Example

Some typical CSS code for styling a button:

```
.button {
  background-color: #0000FF; // Blue
  border: none;
  color: white;
  padding: 15px 32px;
  text-align: center;
  text-decoration: none;
  display: inline-block;
  font-size: 1.5rem;
}
```

If we happen to have a number of buttons on our website, all of which are styled in a similar manner, we would have a good case for inheritance!

We would implement a secondary button via inheritence like so:

```
.button-secondary {
  @extend .button;
  background-color: #4CAF50; // Green
}
```

Our `.button-secondary` class will take on all of the properties and values set the `.button class`, with the exception of `background-color` which we've decided to set to green.

Try it out below!

Output

HTML

CSS (SCSS)

```
.button {
  background-color: #0000FF; // Blue
  border: none;
  color: white;
  padding: 15px 32px;
  text-align: center;
  text-decoration: none;
  display: inline-block;
  font-size: 1.5rem;
}

.button-secondary {
  @extend .button;
  background-color: #4CAF50; // Green
}
```

The use of inheritance helps us to keep our code neat, clean and focused on constructing reusable components.

Up next, we'll take a look at the `&` operator.

The '&' Operator

In this lesson, we'll be looking at the ampersand ('&') operator syntax.

WE'LL COVER THE FOLLOWING

- Definition
- Example

Definition

The ampersand `&` operator is often used when *nesting* in SASS and is an extremely useful feature.

It can be a great time-saver, especially if you happen to be coding your stylesheets with the BEM methodology, or using any systematic naming convention for your class names.

Example

See the following HTML:

```
<button class = "btn btn--red"> Click me! </button>
```

Typical styling would be something like this:

Output

HTML

CSS (SCSS)

```
.btn {
  display: inline-block;
  padding: 15px 32px;
}
.btn--red {
  background-color: #ff0000; // Red
```

```
}
.btn:hover {


  background-color: #fff; // White
}
```

With the `&` operator we can be much more efficient:

Output

HTML

CSS (SCSS)

```
.btn {
  display: inline-block;
  padding: 15px 32px;
  &--red {
    background-color: #ff0000; // Red
  }
  &:hover {
    background-color: #fff; // White
  }
}
```

Notice that we've now nested the child selectors that use the same `.btn` name, represented by the `&` operator. name.

When compiled the `&` operator will be replaced by its enclosing selector name!

Next up we'll learn about control directives.

Control Directives

In this lesson, we'll be looking at SASS control directives, which allow us to introduce conditionals and looping into our stylesheets.

WE'LL COVER THE FOLLOWING

- Definition
- @if and @else
- @for
- @while

Definition

Control directives and **expressions** are used in SASS to include styles only under certain defined conditions.

As a feature, they're quite advanced and are mainly useful in mixins. Common directives include `@if`, `@else`, `@for` and `@while`.

@if and @else

The `@if` and `@else` directives are similar to `if` and `else` statements in JavaScript.

`@if` takes an expression and executes the styles contained within its block — if the evaluation **is not false** (or null).

`@else` will then be checked, if the previous `@if` evaluated to false.

For example:

Output

HTML

CSS (SCSS)

```
@mixin heading($size) {
  @if $size == large {
    font-size: 4rem;
  }
  @else if $size == medium{
    font-size: 3rem;
  }
  @else if $size == small {
    font-size: 2rem;
  }
  @else {
    font-size: 1rem;
  }
}

.h1 {
  @include heading(large);
}

.h6 {
  @include heading(small);
}
```

Here, we are using a **heading** mixin which accepts `$size` as an argument. We can have a different size for each of our headings depending on which value we pass to the mixin.

@for

You can use the `@for` directive to execute a group of statements a specified number of times. Effectively this is a loop.

It has two variations. The first uses the `through` keyword, it'll execute the statements from **start** to **end**, inclusive.

'through' example:

```
@for $i from 1 through 5 {
   .list-#{$i} {
      width: 2px * $i;
   }
}
```

This will produce the following CSS output:

```
.list-1 {
  margin-left: 2px;
}

.list-2 {
  margin-left: 4px;
}

.list-3 {
  margin-left: 6px;
}

.list-4 {
  margin-left: 8px;
}

.list-5 {
  margin-left: 10px;
}
```

If we replace the `through` keyword with `to`, it makes the loop exclusive. The difference being that it won't execute when the variable is equal to **end**.

'to' Example:

```
@for $i from 1 to 5 {
   .list-#{$i} {
      width: 2px * $i;
   }
}
```

Would yield the following CSS:

```
.list-1 {
  margin-left: 2px;
}

.list-2 {
  margin-left: 4px;
}

.list-3 {
```

```
  margin-left: 6px;
}


.list-4 {
  margin-left: 8px;
}
```

@while

We could instead implement the above code using the `@while` directive. As its name implies, it will continue to output CSS produced by the statements while the condition returns true.

The syntax is as follows:

```
$i: 1;
@while $i < 6 {
  .list-#{$i} {
     width: 2px * $i;
  }
  $i: $i + 1;
}
```

The output is identical, so you could opt for your personal preference based on the syntax.

Next, we'll look at SASS interpolation.

Interpolation

In this lesson, we'll learn how to use Interpolation to make our code more dynamic.

WE'LL COVER THE FOLLOWING

- Definition
- Syntax
- Main reasons to use Interpolation

Definition

Interpolation is essentially a code insertion. It allows us to interpolate SASS expressions into our code. We can use it to use a selector or property name, quoted or unquoted strings etc, as variables.

Syntax

To interpolate an expression we need to wrap the expression using `#{ }`.

```
#{$variable_name}
```

Let's see an example that shows how we could use interpolation with a mixin:

```
@mixin interpolation($editable, $val, $val2, $prop1, $prop2)
{
    background-#{$editable}: $val;
    position: $val2;
    #{$prop1}: 0px;
    #{$prop2}: 0px;
}


.block1{
    @include interpolation("image", url("img.png"), absolute, top, right);
}
```

```
.block2{

    @include interpolation("color", lightgray, absolute, top, left);
}
```

This will compile in CSS as follows:

```
.block1 {
    background-image: url("img.png");
    position: absolute;
    top: 0px;
    right: 0px;
}

.block2 {
   background-color: lightgray;
   position: absolute;
   top: 0px;
   left: 0px;
}
```

As you can see, it's quite easy to use this to create dynamically reusable code!

Main reasons to use Interpolation

- We can use dynamically created names as a property name, a variable name or for other similar purposes.
- We can create highly reusable code!

Next, we'll learn how to use placeholders in SASS.

Placeholders

In this lesson, we'll be looking at SASS placeholders.

WE'LL COVER THE FOLLOWING

- For Example:
- Why use Placeholders?

For Example:

In SASS a **placeholder** looks and acts a lot like a class selector, only it starts with a `%` and it's not included in the CSS output.

Our `%placeholder` selector contains some width and height declarations:

```
%placeholder {
    width: 100%;
    height: 100%;
}
body {
    @extend %placeholder;
}
p {
    @extend %placeholder;
}
```

Note that we've used the `@extend` directive, which (as we saw previously) allows one selector to inherit styles of another selector.

This outputs to CSS as follows:

```
body, p {
    width: 100%;
    height: 100%;
}
```

Simple and as expected!

However, the preprocessor will skip `%placeholder` and it won't be included in the final CSS file.

Output

HTML

CSS (SCSS)

```
%placeholder {
    width: 100%;
    height: 100%;
}
body {
    @extend %placeholder;
}
p {
    @extend %placeholder;
}
```

Why use Placeholders?

Placeholder selectors are mainly useful when writing a SASS library where each style rule is optional.

Typically, when working on your own project, it's often better to just extend a class selector instead. But it's good to know as it could come in quite handy if you start working on larger-scale projects.

Next, we'll sum up the features of SASS, before moving on to learn how to structure our projects!

Summing up

Here is a brief summary of what we've covered so far before we move on to the next section!

So far, you've learned what Sass is, how to install and run it on a local server, and we've looked at many of the features that no-doubt make it an extremely useful addition to your front-end skill set!

Using SASS, we write much cleaner code, reduce repetition by reusing code, organize our projects much more effectively and even build programmatic logic into our stylesheets.

SASS makes the maintenance of CSS a much easier task and actually saves us — as developers — a lot of time!

In the next chapter, we'll be looking at how we can go about structuring our SASS projects!

End of section Quiz

1 Which SASS feature lets group together multiple CSS declarations for reuse throughout your project?

How should we structure our Sass projects?

In this next lesson, we'll learn all about structuring our SASS projects, starting with a brief overview.

WE'LL COVER THE FOLLOWING

- Overview
- How do we structure our Sass projects?

Overview

Let's take a look into how we structure our SASS projects.

As projects grow and expand, the need to organize our directory structure increases dramatically. SASS helps us achieve this by giving us the ability to modularize our stylesheet code. We also have the added benefit of creating components that can be reused across multiple projects. Adding a further element of structure & organization.

How do we structure our Sass projects?

We do this by using **Partials** (which we introduced earlier) to divide up our stylesheets into separate files. These files will represent different components.

We then import our partials using an `@import` directive, into one master stylesheet — typically the `main.scss` file.

For example:

```
// File: main.scss

@import 'layout/header';
```

We can then create a layout folder for our layout specific files, such as:

```
// File: _header.scss

// This file contains all styles related to the header of the site/applica
tion.

/* STYLES GO HERE */
```

Remember: The name of a partial file should always begin with an underscore `_`.

Next up, let's take a look at some of the ways we could choose to structure our projects!

Let's keep in mind that there is no one *correct* structure — it's entirely up to you!

Basic structure

In this lesson, we'll take a look at an example of a basic project structure.

If you're using SASS on a small project, such as a single web page, a very minimal project structure could be:

```
_base.scss
_layout.scss
_components.scss
main.scss
```

Here we have 3 partial files connecting up to our `main.scss`.

Base: Contained within this file are all your resets, variables, mixins, and any utility classes.

Layout: Contains all the CSS that handles the layout, such as the container and any grid systems.

Components: Anything reusable such as buttons, navbars, cards etc.

Main: It should **ONLY** contain the imports for the above files.

If any file grows too cluttered or disorganized, it's time to expand our structure. For example, consider adding a new folder for your components, and then break it up into individual files such as `_button.scss` or `_carousel.scss`.

However, when working on a larger project, we'll need a more rigorous architecture, which we'll look at in the next lesson!

The 7–1 Pattern

In this lesson, we'll learn how to organize our projects using the 7-1 pattern.

WE'LL COVER THE FOLLOWING

- The 7-1 pattern
- Abstracts (or utilities)
- Base
- Components (or modules)
- Layout
- Pages
- Themes
- Vendors
- Main.scss

The 7-1 pattern

The architecture known as the **7–1 pattern** (7 folders, 1 file), is a widely-adopted document structure that serves as a basis for large projects.

You have all your partials organised into 7 different folders, and a single file sits at the root level (usually named `main.scss`) to handle the imports — this is the file that compiles into CSS.

What follows is a sample 7–1 directory structure. I've included some examples of files that would sit inside of each folder:

```
sass/
|
|- abstracts/ (or utilities/)
|   |- _variables.scss    // Sass Variables
|   |- _functions.scss    // Sass Functions
|   |- _mixins.scss       // Sass Mixins
```

```
|   |- _mixins.scss        // Sass Mixins
|
|- base/
|   |- _reset.scss         // Reset/normalize
|   |- _typography.scss    // Typography rules
|
|- components/ (or modules/)
|   |- _buttons.scss       // Buttons
|   |- _carousel.scss      // Carousel
|   |- _slider.scss        // Slider
|
|- layout/
|   |- _navigation.scss    // Navigation
|   |- _grid.scss          // Grid system
|   |- _header.scss        // Header
|   |- _footer.scss        // Footer
|   |- _sidebar.scss       // Sidebar
|   |- _forms.scss         // Forms
|
|- pages/
|   |- _home.scss          // Home specific styles
|   |- _about.scss         // About specific styles
|   |- _contact.scss       // Contact specific styles
|
|- themes/
|   |- _theme.scss         // Default theme
|   |- _admin.scss         // Admin theme
|
|- vendors/
|   |- _bootstrap.scss     // Bootstrap
|   |- _jquery-ui.scss     // jQuery UI
|
`- main.scss               // Main Sass file
```

Let’s now look at the typical contents within each folder.

Abstracts (or utilities)

Contains SASS tools, helper files, variables, functions, mixins, and other config files. These files are just meant to be helpers which don’t output any CSS when compiled.

Base

Contains the boilerplate code for the project. Including standard styles such as resets and typographic rules, which are commonly used throughout your

resets and typographic rules, which are commonly used throughout your project.

Components (or modules)

Contains all of your styles for buttons, carousels, sliders, and similar page components (kind of like widgets). Your project will typically contain a lot of component files — as the whole site/app should be mostly composed of small modules.

Layout

Contains all styles involved with the layout of your project. Such as styles for your header, footer, navigation and the grid system.

Pages

Any styles specific to individual pages will sit here. For example, it's not uncommon for the home page of your site to require page specific styles that no other page receives.

Themes

This is likely not used in many projects. It would hold files that create project specific themes. For example, if sections of your site contain alternate color schemes.

Vendors

Vendors contain all third party code from external libraries and frameworks — such as Normalize, Bootstrap, jQueryUI, etc. However, there is often a need to override vendor code. If this is required, it's good practice to create a new folder called vendors-extensions/ and then name any files after the vendors they overwrite. The file `vendors-extensions/_bootstrap.scss` would contain all your Bootstrap overrides — as editing the vendor files directly isn't generally a good idea.

Main.scss

This file should only contain your imports!

For example:

```
@import 'abstracts/variables';
@import 'abstracts/functions';
@import 'abstracts/mixins';

@import 'vendors/bootstrap';
@import 'vendors/jquery-ui';

@import 'base/reset';
@import 'base/typography';

@import 'layout/navigation';
@import 'layout/grid';
@import 'layout/header';
@import 'layout/footer';
@import 'layout/sidebar';
@import 'layout/forms';

@import 'components/buttons';
@import 'components/carousel';
@import 'components/slider';
@import 'components/images';

@import 'pages/home';
@import 'pages/about';
@import 'pages/contact';

@import 'themes/theme';
@import 'themes/admin';
```

Note: There's no need to include the `_` or `.scss` file extension when importing.

Get up and running with 7–1:

Instead of creating this structure manually, you can grab the official boilerplate from Github.

You can download or clone it with the following terminal command:

```
git clone https://github.com/HugoGiraudel/sass-boilerplate.git
```

And that's it! You've learned how you can go about structuring your SASS

projects.

The thing to keep in mind is that there are no explicit rules. You should always structure your projects in a way that is meaningful to you (and your team!). The way that helps you quickly and easily find and isolate your styles is the way to go!

In the next section, we're going to finally learn how to setup & run the SASS build process.

Quick Quiz!

1

In the 7-1 pattern, which folder would ideally contain the files for our mixins?

COMPLETED 0%

1 of 2

Creating a Build Process

In this section, we'll learn how to create a SASS build process, starting with a brief overview.

WE'LL COVER THE FOLLOWING

- What is a build process?
- The Build Process

We will look at how we can use npm scripts to set up a Sass build process and boost our development workflow!

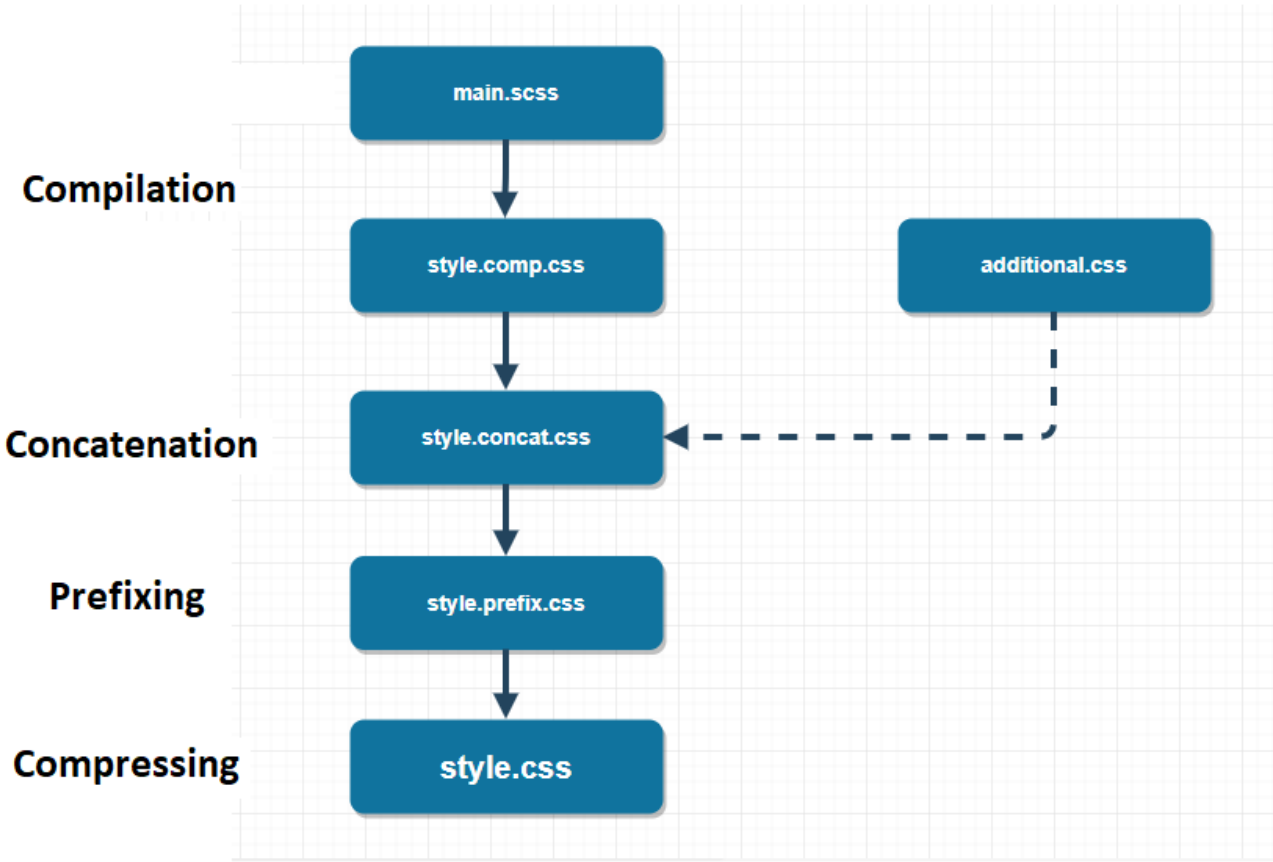
What is a build process?

A build process is essentially just a sequence of tasks that perform automatically — which we run after project completion. Our production files are generated and are then ready to be deployed to a web server.

The Build Process

Firstly, let's break down the steps involved. We will be **compiling**, **concatenating**, **prefixing** and **compressing** our stylesheets. As shown in the

below diagram:



Over the next few lessons, we'll be looking at each step in detail.

The `main.scss` file is our main SASS file which performs the **compilation** to CSS.

Next we'll take a look at **concatenation**. In this process, we want to merge all of our CSS files into one file. To test out this process, We'll use an additional CSS file called `additional.css`.

Then we'll look at **prefixing** with an autoprefixer.

As a refresher, sometimes when working with CSS we'll use nonstandard properties. Browser vendors add prefixes for these experimental properties, so they can be used before they're standardized.

When we use an autoprefixer, any vendor prefixes (-webkit, -moz, etc) will be automatically added to our code, to help ensure its functionality across the major browsers.

And the final step in our process will be **compression**, where we compress all our code to maximize performance.

So let's go ahead and create our build process with npm scripts!

Compilation

In this lesson, we'll learn how to setup a compilation script and run our watch script to check for file changes.

Before beginning, make sure to have your SASS project directory open! Then, open your `package.json` file. This is the file that generates when you run `npm init` in your Sass project directory.

In `package.json`, add the following code:

```
"compile-sass": "node-sass sass/main.scss css/style.comp.css"
```

Your script should now be as follows:

```
"scripts": {
  "watch-sass": "node-sass sass/main.scss css/style.css --watch",
  "compile-sass": "node-sass sass/main.scss css/style.comp.css"
},
```

Remember we added the `--watch` flag previously, in the lesson How to Install and Configure SASS.

Now let's ensure the compile is working:

In a terminal, open up to your Sass project folder and run:

```
npm run compile-sass
```

It should complete the render and output the **style.comp.css** file to your CSS folder.

We will run this task at the end of the project to complete our final build!

While we are developing the project, we run the watch task with:

```
npm run watch-sass
```

This tells the compiler to watch the source files for changes, and re-compile to CSS automatically, whenever you save your Sass files. Just be sure to keep the task running while you work!

In the next lesson, we'll look at the concatenation process.

Concatenation

In this lesson, we'll learn how to concatenate our files.

The next step is to add a script to concatenate our existing CSS files.

As mentioned previously, I've created the `additional.css` file for our merge. Inside of `additional.css`, there are a few additional styles.

Go ahead and also create a new CSS file in your CSS folder. Just give it some additional styles — it doesn't matter what.

We'll be adding the following line of code to the end of our script:

```
"concat-css": "concat -o css/style.concat.css css/additional.css css/styl
e.comp.css"
```

Our `package.json` will now be as follows:

```
"scripts": {
  "watch-sass": "node-sass sass/main.scss css/style.css --watch",
  "compile-sass": "node-sass sass/main.scss css/style.comp.css",
  "concat-css": "concat -o css/style.concat.css css/additional.css css/sty
le.comp.css"
},
```

Let's break down the line of code we just added:

`concat-css:` is our script name.

`concat -o css/style.concat.css` is our package output file.

`css/additional.css css/style.comp.css` are our CSS files to be concatenated.

In order for this to run, we'll need to install the concat npm package, run the following command:

```
npm install concat --save-dev
```

```
npm install concat --save-dev
```

Once the install completes, you'll see it listed under the ***devDependencies*** of your `package.json`.

Now, let's run the concat to make sure it works:

```
npm run concat-css
```

You'll now see the `style.concat.css` output file in your CSS directory!

Open it up and take a look at your CSS, you'll see that the contents of your `additional.css` and your `style.comp.css` have merged into one!

Next up, we'll take a look at how to automatically prefix our code.

Prefixing

In this lesson, we'll learn how to setup autoprefixing.

Let's now move on to adding prefixing into our build.

Recall that in CSS you'll often need to prefix your code in order to ensure cross-browser compatibility. Like in the following example:

```
.myClass {
    -webkit-transition: all 10s linear;
    -moz-transition: all 10s linear;
    -ms-transition: all 10s linear;
    -o-transition: all 10s linear;
    transition: all 10s linear;
}
```

You could do this manually, or use a tool like Autoprefixer, which automates the process entirely. So we don't have to check if a prefix is required.

Let's add autoprefixing into our build process by adding the following line to the end of the script:

```
"prefix-css": "postcss --use autoprefixer -b 'last 5 versions' css/style.c
oncat.css -o css/style.prefix.css"
```

The script so far should look like so:

```
"scripts": {
  "watch-sass": "node-sass sass/main.scss css/style.css --watch",
  "compile-sass": "node-sass sass/main.scss css/style.comp.css",
  "concat-css": "concat -o css/style.concat.css css/additional.css css/sty
le.comp.css",
  "prefix-css": "postcss --use autoprefixer -b 'last 5 versions' css/styl
e.concat.css -o css/style.prefix.css"
},
```

So, we `npm run watch` in our script

So here, `prefix-css` is our script name.

`postcss --use autoprefixer` autoprefixer is selected.

`-b 'last 5 versions'` specifies which browser versions we want our autoprefixes to cover (in this case the last 5).

`css/style.concat.css` is our input file.

`-o css/style.prefix.css` is our specified output file.

We're using the npm autoprefixer package, which will need to be installed by running:

```
npm install autoprefixer --save-dev
```

We also need to install PostCSS (autoprefixer is part of this plugin). We use the following command:

```
npm install postcss-cli --save-dev
```

And then run the script as follows:

```
npm run prefix-css
```

It will generate our `css/style.prefix.css` file.

Take a look at the code in this file; you'll see the browser prefixes have been automatically added for you.

This is great as we can now forget about prefixing and concentrate on writing clean code!

In the next lesson, we'll add compression to our process.

Compressing

In this lesson, we'll learn how to add compression into our process in order to fully-optimize our code before it's released into the world!

Add the following line to our script:

```
"compress-css": "node-sass css/style.prefix.css css/style.css --output-sty
le compressed"
```

And our `package.json` now looks like so:

```
"scripts": {
  "watch-sass": "node-sass sass/main.scss css/style.css --watch",
  "compile-sass": "node-sass sass/main.scss css/style.comp.css",
  "concat-css": "concat -o css/style.concat.css css/additional.css css/sty
le.comp.css",
  “prefix-css”: “postcss --use autoprefixer -b 'last 5 versions' css/styl
e.concat.css -o css/style.prefix.css”,
  "compress-css": "node-sass css/style.prefix.css css/style.css --output-s
tyle compressed"
},
```

This is a nice easy one! Here all we do is tell our `css/style.prefix.css` input file to output to `css/style.css`. The `--output-style compressed` option will compress the code!

Let's test it out...

```
npm run compress-css
```

Now, take a look at your `style.css` file.

You'll see that all of your styles have compressed into a single line of code! All white-space and comments have been removed.

You can compare the file size of your `style.prefix.css` input file with the

newly generated `style.css` file, to contrast with the compressed file size.

With this simple step, we have just significantly reduced our page load!

Next up we'll bring the entire process together with one final script.

The Build!

In this final step, we'll add one more script, which will execute the entire process!

WE'LL COVER THE FOLLOWING

- Final build
- You can use this build on any future projects!

Final build

Lets now write one final script to run everything at once!

Add the following line of code:

```
"build-css": "npm-run-all compile-sass concat-css prefix-css compress-css"
```

And our final `package.json` looks like so:

```
"scripts": {
  "watch-sass": "node-sass sass/main.scss css/style.css --watch",
  "compile-sass": "node-sass sass/main.scss css/style.comp.css",
  "concat-css": "concat -o css/style.concat.css css/additional.css css/sty
le.comp.css",
  “prefix-css”: “postcss --use autoprefixer -b 'last 5 versions' css/styl
e.concat.css -o css/style.prefix.css”,
  "compress-css": "node-sass css/style.prefix.css css/style.css --output-s
tyle compressed",
  "build-css": "npm-run-all compile-sass concat-css prefix-css compress-cs
s"
},
```

Here, we’ve simply added all our tasks `compile-sass`, `concat-css`, `prefix-css` & `compress-css` to be run when we execute our build command.

We use the npm-run-all package to ensure it works on all platforms.

Enter the following command:

```
npm install npm-run-all --save-dev
```

Let's run a final test to confirm everything is working!

Go ahead and delete all of the files (except `additional.css`), from the CSS folder.

Once that's done, run the build command:

```
npm run build-css
```

And there you go! All of your CSS files have been generated with this one command — powerful stuff!

You can use this build on any future projects!

To set up this build on future projects, you only need to copy the scripts and ***devDependencies*** from this project into the `package.json` of your new project and run `npm install`.

We have now created a build process for our SASS projects! We can compile, merge, prefix and compress our stylesheets with a single command.

Thus, we've significantly improved upon our development workflow!

Next up, I've included a bonus section on how we can use SASS mixins to better manage our Media Queries.

End of section Quiz!

1 What does prefixing achieve?

How to setup Media Queries with SASS Mixins

In this lesson, we'll learn how to use SASS Mixins to make managing Media Queries a breeze.

WE'LL COVER THE FOLLOWING

- CSS Media Queries
- Mixins
- Setting up your mixins
- Using a mixin
- Another way to setup our mixins

When building websites and apps, we'll often use media queries to help ensure our layouts' responsivity. This way, they can adapt to suit the multitudes of devices accessing them, as we need to ensure all of our users are catered for when engaging with our products.

CSS Media Queries

If you have experience with CSS, you'll likely know that a media query is a CSS3 module which allows content to render and adapt to conditions. Such as screen resolution, for example:

```
@media (max-width: 599px) {
 font-size: 1rem;
}
```

Here, we've set the font size to `1rem` when the viewport size is <= `599px`.

This is simple enough, and of course we'll need multiple media queries to have a fully responsive site.

However, inevitably, the time will come where we'll need to edit the rules for the actual breakpoints, and searching through all of your code to change every line affected by a rule change is far from ideal. There has to be a better way!

Modern layout specifications have improved greatly over the past few years, Grid and Flexbox are responsive by default. This helps us to cut down on the amount of media queries required in our projects & makes for cleaner code.

However, media queries still have a place in modern web development. And as our projects grow bigger, we need a better way of managing them.

Enter SASS mixins!

Mixins

As we now know, SASS mixins give us the ability to create reusable chunks of code — they reduce repetition, promote dry code and allow for ease of maintenance.

So writing media queries as mixins to inject into your stylesheets, wherever they're required makes a whole lot of sense!

Let's now take a look at an example.

Setting up your mixins

```
@mixin for-phone-only {
  @media (max-width: 599px) { @content; }
}
@mixin for-tablet-portrait-up {
  @media (min-width: 600px) { @content; }
}
@mixin for-tablet-landscape-up {
  @media (min-width: 900px) { @content; }
```

```
}
@mixin for-desktop-up {
  @media (min-width: 1200px) { @content; }
}
@mixin for-big-desktop-up {
  @media (min-width: 1800px) { @content; }
}
```

Here, we've written up 5 common breakpoints into `@mixin` blocks.

The `@content` syntax is a directive of SASS which allows for the insertion of content later on, as you likely won't be writing all of your queries at once.

Using a mixin

Say we want to use a mixin to reduce the font size of our header text on mobile devices.

We add the mixin (nested under the appropriate class) as an **include**, like so:

```
.header-title {
   font-size: 2rem;

   @include for-phone-only {
      font-size: 1rem;
   }
}
```

When we compile our project, our @include will translate into:

```
@media (max-width: 599px) {
   font-size: 1rem;
}
```

We now have the ability to easily insert our media queries wherever we like throughout our projects!

Additionally, we won't need to memorize our breakpoints, as we have them pre-defined. If we need to alter them, we can simply edit the mixins.

Another way to setup our mixins

If you want to take it a step further, you could use conditionals to setup your

mixins.

You create just one `@mixin` block and set the parameters to be passed in — which we use to select our breakpoints, like so:

```
@mixin for-size($size) {
  @if $size == phone-only {
    @media (max-width: 599px) { @content; }
  } @else if $size == tablet-portrait-up {
    @media (min-width: 600px) { @content; }
  } @else if $size == tablet-landscape-up {
    @media (min-width: 900px) { @content; }
  } @else if $size == desktop-up {
    @media (min-width: 1200px) { @content; }
  } @else if $size == big-desktop-up {
    @media (min-width: 1800px) { @content; }
  }
}
```

Then, to use our mixins in this manner, we'd select it like so:

```
.header-title {
   font-size: 2rem;

   @include for-size(phone-only) {
      font-size: 1rem;
   }
}
```

It's that simple!

Writing media queries doesn't need to be a headache, and by using SASS mixins, we have one centralized location in which to manage our media queries.

Handling the responsivity of our projects just got a whole lot easier!

That's it! Congratulations for completing the course! You're now equipped with all the knowledge and tools required to implement SASS effectively, on any or all of your future projects.

Keep that momentum going & use it on the next one!

– Tim

- Hint

How to Install and Configure SASS Locally

In this lesson, we'll cover the installation of SASS onto our local machine. Including the initial configuration & live server setup.

WE'LL COVER THE FOLLOWING

- Environment setup
- Folder Structure
- File Structure
- Initializing our Project Directory
- Compiling Sass Code to CSS
- Live Reload

Before we can write Sass code, it needs to be installed locally. As by default, it's not a language known to the browser.

Let's now go through the process to setup the environment that will allow us to write then compile Sass.

Note: When Sass is compiled, it is converted into regular CSS code that browsers can interpret and render.

Environment setup

Before we start, you must have **npm** installed on your computer, it comes bundled with Node.js; you can install it from here. Go ahead and install it if you haven't already.

If you are unsure whether you have Node.js installed or not, run `node -v` from your terminal.

If you see a version number, it's installed!

A note on terminal:

If you are new to SASS, chances are you may also be new to running commands from the terminal. It's not as daunting as it might seem! And it's a real time-saver once you gain more experience.

To open a terminal on a Windows PC, right-click the Windows Icon and select *Windows Powershell*, if you're on a Mac go to *Finder* > *Applications* > *Utilities* > *Terminal*.

Folder Structure

Let's create our project folders! They will be structured like so:

```
sass-project
   |- sass
   |- css
```

To create this structure, open terminal and change to the folder you wish to install the sass project into (via the cd command).

Then run the following commands:

```
mkdir sass-project
cd sass-project
mkdir -p sass css
```

File Structure

You will need an **index.html** and **main.scss** in this folder.

To create these files, run:

```
touch index.html
cd sass
touch main.scss
cd ..
```

You'll also need a default CSS stylesheet for the SASS to compile into:

```
cd css
touch style.css
```

Open up your index.html and paste in the following code:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Index page</title>
    <link rel="stylesheet" href="css/style.css">
  </head>
  <body>

  </body>
</html>
```

This is just our boilerplate code with the stylesheet connected!

Initializing our Project Directory

All projects that use npm need to be initialized. To do this, ensure you're still in the sass-project folder and run the following command:

```
npm init -y
```

This will create a **package.json** file for our project. We'll be learning more about the configuration of this file later in the course!

Install node-sass

node-sass is the library which allows us to compile `.scss` to `.css`.

Run the following command to install node-sass as *dev dependency*.

```
npm install node-sass --save-dev
```

Note: A **dev dependency** is only used in the build phase of our project. It's not included at runtime.

Compiling Sass Code to CSS

Next, we need to create an *npm script* to run the compilation.

Add this script inside the script section of our previously created **package.json** file:

```
"compile-sass": "node-sass sass/main.scss css/style.css"
```

Don't forget to separate each script with a comma!

We have here specified **main.scss** as our main Sass file and **style.css** as the compiled CSS file.

To compile our SASS code into CSS, all we need to do is run:

```
npm run compile-sass
```

Live Reload

Let's also add a live reload to our project! To do this run the following to install globally:

```
npm install live-server -g
```

Now, make sure you're still in the Sass project folder, and run:

```
live-server
```

And just like that, you've got a pretty neat dev environment with your project running locally on HTTP.

You'll need to keep `live-server` and `npm run compile-sass` running in two separate terminal windows.

So we now have the project environment all set up on the local machine!